causal forest

2026-09-30

Setup

```
required_packages <- c("haven", "estimatr", "dplyr", "modelsummary")
missing_packages  <- setdiff(required_packages, rownames(installed.packages()))
if (length(missing_packages) > 0) {
  install.packages(missing_packages, repos = "https://cloud.r-project.org")
}
library(haven)
library(estimatr)
```

```
## Warning: package 'estimatr' was built under R version 4.4.3
```

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(modelsummary)
```

```
## Warning: package 'modelsummary' was built under R version 4.4.3
```

```
library(tidyr)
library(grf)
```

```
## Warning: package 'grf' was built under R version 4.4.3
```

```
library(knitr)


data_path <- file.path("data", "oreopoulos-resume-study-replication-data-file.dta")
if (!file.exists(data_path)) {
  stop("Data file not found. Expected it at: ", normalizePath(data_path, mustWork = FALSE))
}
df <- read_dta(data_path)
```

```r
vars <- c("type", "female", "ba_quality", "extracurricular_skills", "language_skills",
          "ma", "same_exp", "exp_highquality", "reference", "accreditation", "legal")

df_int1 <- df %>%
  filter(
    type %in% c(0, 1),
    name_ethnicity %in% c("Canada", "Chinese", "Indian", "Pakistani")
  )
```

Check for missing values across waves (fall_data)

```r
counts_all <- df_int1 %>%
  mutate(across(all_of(vars), as.character)) %>%
  pivot_longer(all_of(vars), names_to = "variable", values_to = "value") %>%
  mutate(variable = factor(variable, levels = vars)) %>%
  group_by(fall_data, variable) %>%
  summarise(
    with_value = sum(!is.na(value) & trimws(value) != ""),
    empty      = sum(is.na(value) | trimws(value) == ""),
    .groups = "drop"
  ) %>%
  left_join(df_int1 %>% count(fall_data, name = "total_rows"), by = "fall_data") %>%
  relocate(total_rows, .after = fall_data) %>%
  arrange(fall_data, variable)

print(counts_all, n = Inf)
```

```
## # A tibble: 33 x 5
##    fall_data total_rows variable               with_value empty
##        <dbl>      <int> <fct>                       <int> <int>
##  1         0       1902 type                         1902     0
##  2         0       1902 female                       1902     0
##  3         0       1902 ba_quality                   1902     0
##  4         0       1902 extracurricular_skills       1902     0
##  5         0       1902 language_skills              1902     0
##  6         0       1902 ma                           1902     0
##  7         0       1902 same_exp                     1902     0
##  8         0       1902 exp_highquality              1902     0
##  9         0       1902 reference                       0  1902
## 10         0       1902 accreditation                   0  1902
## 11         0       1902 legal                           0  1902
## 12         1       1495 type                         1495     0
## 13         1       1495 female                       1495     0
## 14         1       1495 ba_quality                   1495     0
## 15         1       1495 extracurricular_skills       1495     0
## 16         1       1495 language_skills              1495     0
## 17         1       1495 ma                           1495     0
## 18         1       1495 same_exp                     1495     0
## 19         1       1495 exp_highquality              1495     0
## 20         1       1495 reference                       0  1495
```

```
## 21         1       1495 accreditation                  0  1495
## 22         1       1495 legal                          0  1495
## 23         2       2260 type                        2260     0
## 24         2       2260 female                      2260     0
## 25         2       2260 ba_quality                  2260     0
## 26         2       2260 extracurricular_skills      2260     0
## 27         2       2260 language_skills             2260     0
## 28         2       2260 ma                          2260     0
## 29         2       2260 same_exp                       0  2260
## 30         2       2260 exp_highquality             2260     0
## 31         2       2260 reference                   2260     0
## 32         2       2260 accreditation               2260     0
## 33         2       2260 legal                       2260     0
```

```r
df_int1 <- df_int1 %>%
  mutate(
    D              = as.numeric(type == 1),   # 1 = foreign name (Type 1), 0 = Type 0 baseline
    fall_data      = factor(fall_data),
    female                 = ifelse(is.na(female), 0, female),
    ba_quality              = ifelse(is.na(ba_quality), 0, ba_quality),
    extracurricular_skills  = ifelse(is.na(extracurricular_skills), 0, extracurricular_skills),
    language_skills         = ifelse(is.na(language_skills), 0, language_skills),
    ma                      = ifelse(is.na(ma), 0, ma),
    same_exp                = ifelse(is.na(same_exp), 0, same_exp),
    exp_highquality         = ifelse(is.na(exp_highquality), 0, exp_highquality)
    # Reference, accreditation, and legal variables are only available in some waves
    # (present only when fall_data == 2), so drop them to ensure consistency across waves.
  )

cat("Analysis sample size:", nrow(df_int1), "\n")
```

```
## Analysis sample size: 5657
```

```r
cat("Treatment (D) distribution:\n")
```

```
## Treatment (D) distribution:
```

```r
print(table(df_int1$D))
```

```
##
##    0    1
## 3026 2631
```

Causal forest 1 (3 covariates)

1. Cluster by firmid
2. X (Covariates): type, language, extracurricular, multinational
3. Y: callback rate
4.

```r
set.seed(4101)  # For reproducibility
forest1df_X <- df_int1 %>%
  select(language_skills,extracurricular_skills,same_exp)%>%
  as.matrix()

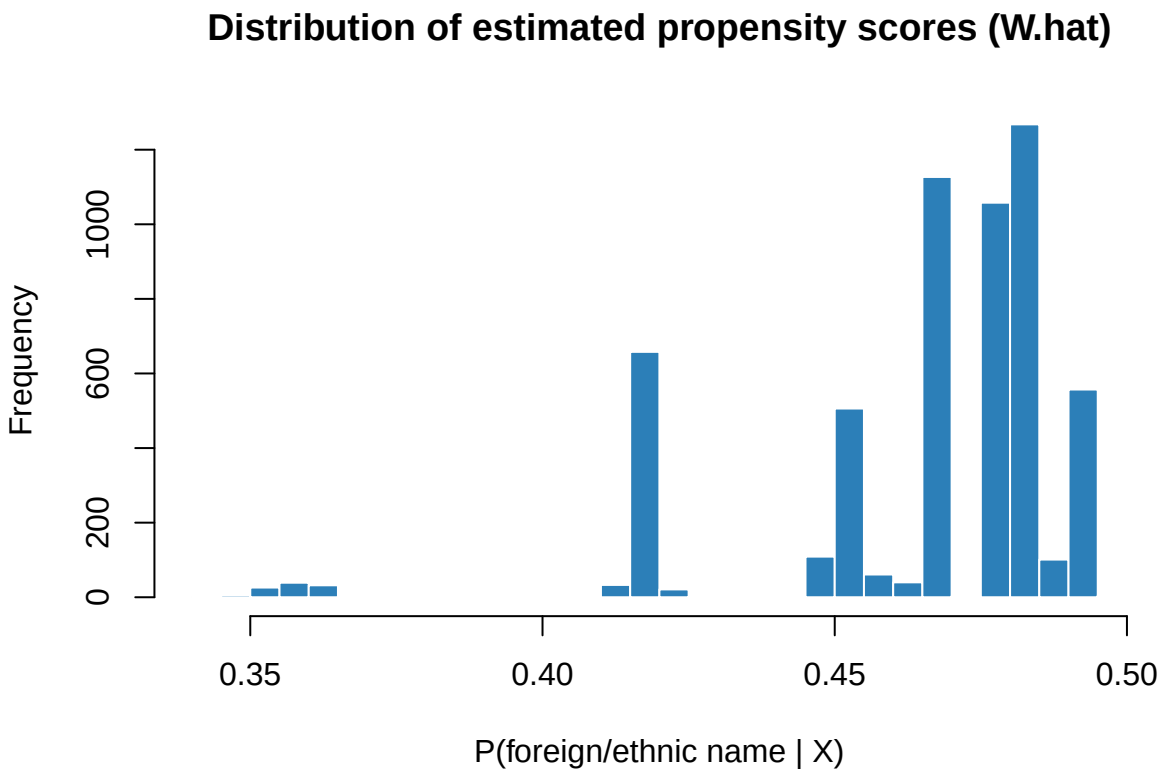
forest1df_treatment <- df_int1$D %>% as.matrix()
forest1df_cluster<-df_int1$firmid %>% as.matrix()
forest1df_Y<-df_int1$callback %>% as.matrix()

cf1<-causal_forest(forest1df_X,
                   forest1df_Y,
                   forest1df_treatment,
                   clusters=forest1df_cluster)
```

Overlap / propensity check

Because name ethnicity is experimentally randomized within a job posting, the estimated propensity scores should cluster fairly tightly around the actual treatment share rather than displaying the extreme values that signal confounding.

```r
hist(cf1$W.hat,
     breaks = 30,
     col = "#2c7fb8",
     border = "white",
     main = "Distribution of estimated propensity scores (W.hat)",
     xlab = "P(foreign/ethnic name | X)")
```



Average treatment effect

```
ate <- average_treatment_effect(cf1, target.sample = "all")
ate
```

```
##     estimate      std.err
## -0.040919053  0.006878686
```

```
cat(sprintf(
  "**Estimated ATE:** having a foreign/ethnic-sounding name changes the callback probability by **%.3f*
  100 * ate["estimate"], 100 * ate["std.err"]
))
```

Estimated ATE: having a foreign/ethnic-sounding name changes the callback probability by **-4.092** percentage points (SE = 0.688), on average across the analysis sample.

Variable importance

Which of the three moderators does the forest use most to split on (i.e., which one drives the most predictable heterogeneity)?

```r
vi <- variable_importance(cf1)
vi_tab <- data.frame(
  variable = colnames(forest1df_X),
  importance = as.numeric(vi)
) %>% arrange(desc(importance))

kable(vi_tab, digits = 3, caption = "Causal forest variable importance")
```

Table 1: Causal forest variable importance

variable	importance
extracurricular_skills	0.370
same_exp	0.311
language_skills	0.275

Causal forest 2 (ALL covariates)

1. Cluster by firmid
2. X (Covariates): ALL
3. Y: callback rate
4.

```r
# TO ASK: MUST WE INCLUDE FALL_DATA?
set.seed(4101)  # For reproducibility
df_int1 <- df_int1 %>%
  mutate(
    fall_data = as.numeric(as.character(fall_data))
  )


all_pretreat_cov <- c(
  "language_skills",
  "female",
  "ba_quality",
  "extracurricular_skills",
  "ma",
  "same_exp",
  "exp_highquality"
)


all_pretreat_cov_fall <- c(
  "language_skills",
  "female",
  "ba_quality",
  "extracurricular_skills",
  "ma",
  "same_exp",
  "exp_highquality",
  "fall_data"
)
```

```r
forest2df_X <- df_int1 %>%
  select(all_of(all_pretreat_cov_fall))%>%
  as.matrix()

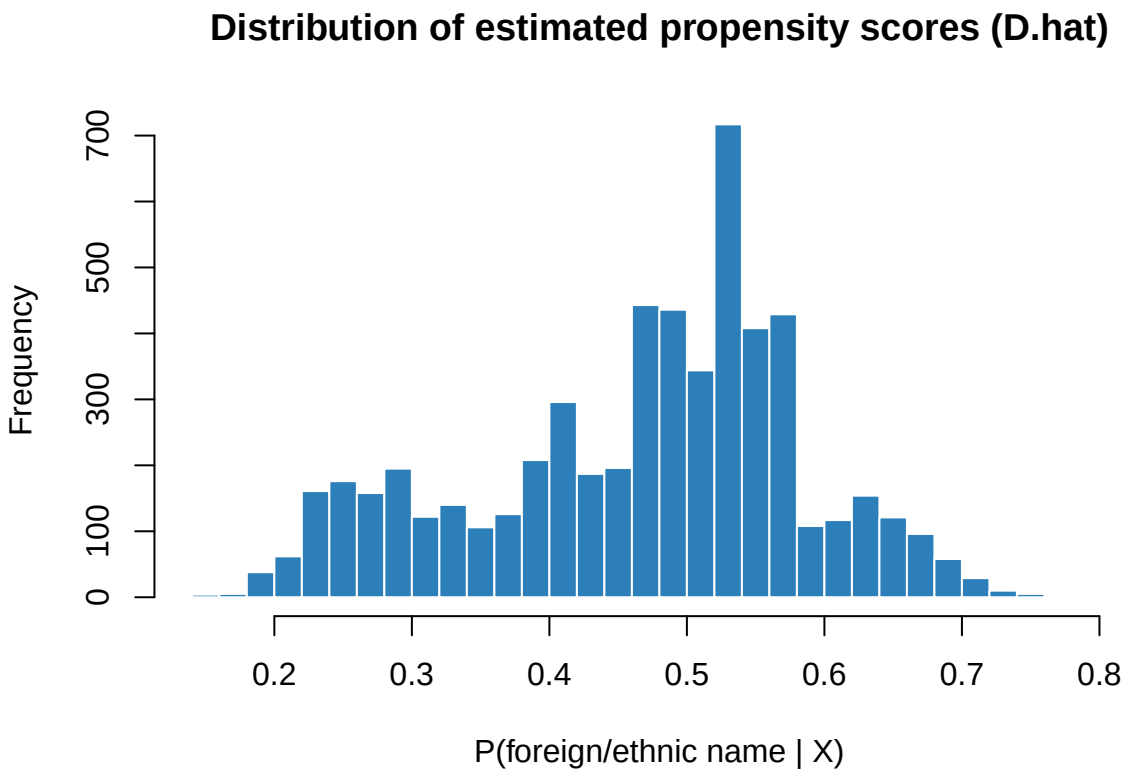
forest2df_treatment <- df_int1$D %>% as.matrix()
forest2df_cluster<-df_int1$firmid %>% as.matrix()
forest2df_Y<-df_int1$callback %>% as.matrix()

cf2<-causal_forest(forest2df_X,
                   forest2df_Y,
                   forest2df_treatment,
                   clusters=forest2df_cluster)
```

Overlap / propensity check

Because name ethnicity is experimentally randomized within a job posting, the estimated propensity scores should cluster fairly tightly around the actual treatment share rather than displaying the extreme values that signal confounding.

```r
hist(cf2$W.hat,
     breaks = 30,
     col = "#2c7fb8",
     border = "white",
     main = "Distribution of estimated propensity scores (D.hat)",
     xlab = "P(foreign/ethnic name | X)")
```



Average treatment effect

```
ate <- average_treatment_effect(cf2, target.sample = "all")
ate
```

```
##     estimate      std.err
## -0.044457714  0.007410193
```

```
cat(sprintf(
  "**Estimated ATE:** having a foreign/ethnic-sounding name changes the callback probability by **%.3f*
  100 * ate["estimate"], 100 * ate["std.err"]
))
```

Estimated ATE: having a foreign/ethnic-sounding name changes the callback probability by **-4.446** percentage points (SE = 0.741), on average across the analysis sample.

Variable importance

Which of the three moderators does the forest use most to split on (i.e., which one drives the most predictable heterogeneity)?

```r
vi <- variable_importance(cf2)
vi_tab <- data.frame(
  variable = colnames(forest2df_X),
  importance = as.numeric(vi)
) %>% arrange(desc(importance))

kable(vi_tab, digits = 3, caption = "Causal forest variable importance")
```

Table 2: Causal forest variable importance

variable	importance
female	0.203
ba_quality	0.194
extracurricular_skills	0.134
language_skills	0.114
exp_highquality	0.098
fall_data	0.093
same_exp	0.086
ma	0.079

0. Sanity checks on sample and treatment coding

```r
library(estimatr)

cat("N:", nrow(df_int1), "(expected 5,657)\n")
```

```
## N: 5657 (expected 5,657)
```

```r
cat("Firms:", length(unique(df_int1$firmid)), "\n")
```

```
## Firms: 3269
```

```r
cat("Resumes per firm:\n")
```

```
## Resumes per firm:
```

```r
print(summary(as.numeric(table(df_int1$firmid))))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    1.00    1.00    2.00    1.73    2.00    2.00
```

```r
cat("\nTreatment counts (expected 3,026 control / 2,631 treated):\n")
```

```
##
## Treatment counts (expected 3,026 control / 2,631 treated):
```

```r
print(table(D = df_int1$D))
```

```
## D
##    0    1
## 3026 2631
```

```r
cat("\nName ethnicity by treatment (D must match the coding exactly):\n")
```

```
##
## Name ethnicity by treatment (D must match the coding exactly):
```

```r
print(table(ethnicity = as.character(df_int1$name_ethnicity), D = df_int1$D))
```

```
##            D
## ethnicity     0    1
##   Canada   3026    0
##   Chinese     0  998
##   Indian      0 1167
##   Pakistani   0  466
```

```r
cat("\nWave by treatment:\n")
```

```
##
## Wave by treatment:
```

```r
print(table(wave = df_int1$fall_data, D = df_int1$D))
```

```
##     D
## wave    0    1
##    0  953  949
##    1 1029  466
##    2 1044 1216
```

`cf1w` is the three-moderator forest plus the wave indicator, so it is the like-for-like comparison with the wave-adjusted regressions. `cf1` and `cf2` are the forests as already built. All three are validated and reported; none is picked after the fact.

```r
set.seed(4101)  # For reproducibility
forest1wX <- df_int1 %>%
  select(language_skills, extracurricular_skills, same_exp, fall_data) %>%
  as.matrix()

cf1w <- causal_forest(forest1wX,
                      as.numeric(df_int1$callback),
                      df_int1$D,
                      clusters = df_int1$firmid)

forests <- list(
  "cf1 (3 moderators)"         = list(cf = cf1,  X = forest1df_X),
  "cf1w (3 moderators + wave)" = list(cf = cf1w, X = forest1wX),
  "cf2 (full covariates)"      = list(cf = cf2,  X = forest2df_X)
)
```

2. Balance table (does not depend on any forest)

SMD = (mean in D=1 minus mean in D=0) / sqrt((var in D=1 + var in D=0) / 2). Rule of thumb: |SMD| < 0.1 counts as balanced. The p-value is from a firm-clustered regression of the variable on D. Wave is a three-level code, so it enters as one dummy per wave rather than as a numeric code.

```
smd <- function(x, d) {
  (mean(x[d == 1]) - mean(x[d == 0])) /
    sqrt((var(x[d == 1]) + var(x[d == 0])) / 2)
}

fall_num <- as.numeric(as.character(df_int1$fall_data))
waves    <- sort(unique(fall_num))

df_bal <- df_int1
for (k in waves) df_bal[[paste0("wave_", k)]] <- as.numeric(fall_num == k)

bal_vars <- c(all_pretreat_cov, paste0("wave_", waves))

balance_row <- function(v, dat) {
  x   <- as.numeric(dat[[v]])
  p   <- tryCatch({
    fit <- lm_robust(as.formula(paste(v, "~ D")), data = dat,
                     clusters = firmid, se_type = "CR2")
    unname(fit$p.value["D"])
  }, error = function(e) NA_real_)
  data.frame(variable = v,
             mean_D0  = mean(x[dat$D == 0]),
             mean_D1  = mean(x[dat$D == 1]),
             SMD      = smd(x, dat$D),
             p_clustered = p)
}

bal_tab <- bind_rows(lapply(bal_vars, balance_row, dat = df_bal)) %>%
  mutate(flag = ifelse(abs(SMD) >= 0.1, "check", ""))

kable(bal_tab, digits = 3,
      caption = "Balance of resume characteristics by name type (D = 1 foreign name)")
```

Table 3: Balance of resume characteristics by name type (D = 1 foreign name)

variable	mean_D0	mean_D1	SMD	p_clustered	flag
language_skills	0.247	0.257	0.022	0.384	
female	0.510	0.506	-0.008	0.767	
ba_quality	0.590	0.636	0.095	0.000	
extracurricular_skills	0.600	0.608	0.016	0.593	
ma	0.205	0.198	-0.016	0.547	
same_exp	0.284	0.243	-0.093	0.000	
exp_highquality	0.331	0.318	-0.028	0.328	
wave_0	0.315	0.361	0.097	0.000	
wave_1	0.340	0.177	-0.379	0.000	check
wave_2	0.345	0.462	0.241	0.000	check

Wave checks (paste right after the val-balance chunk)

The balance table shows treated share differs a lot by wave. These three chunks check whether that is a composition issue (which name types were sampled in which wave) or a randomisation problem within wave, and whether the name penalty itself differs by wave.

```
# Which name types appear in which wave?
print(table(wave = df_int1$fall_data,
            ethnicity = as.character(df_int1$name_ethnicity)))
```

```
##     ethnicity
## wave Canada Chinese Indian Pakistani
##    0    953     481    468         0
##    1   1029       0      0       466
##    2   1044     517    699         0
```

```
# Treatment share and callback rate by wave
wave_tab <- df_int1 %>%
  group_by(wave = fall_data) %>%
  summarise(n = n(),
            treated_share    = mean(D),
            callback_rate_D0 = mean(as.numeric(callback)[D == 0]),
            callback_rate_D1 = mean(as.numeric(callback)[D == 1]),
            .groups = "drop")

kable(wave_tab, digits = 3, caption = "Treatment share and callback rate by wave")
```

Table 4: Treatment share and callback rate by wave

wave	n	treated_share	callback_rate_D0	callback_rate_D1
0	1902	0.499	0.158	0.115
1	1495	0.312	0.127	0.079
2	2260	0.538	0.141	0.098

```
# Balance within wave: difference between name types after absorbing wave fixed effects
within_row <- function(v) {
  dat   <- df_int1
  dat$x <- as.numeric(dat[[v]])
  fit   <- lm_robust(x ~ D + factor(fall_data), data = dat,
                     clusters = firmid, se_type = "CR2")
  data.frame(variable         = v,
             diff_D1_minus_D0 = unname(coef(fit)["D"]),
             se               = unname(fit$std.error["D"]),
             p_clustered      = unname(fit$p.value["D"]))
}

within_tab <- bind_rows(lapply(all_pretreat_cov, within_row))

kable(within_tab, digits = 3,
      caption = "Balance within wave (name-type difference, wave fixed effects, firm-clustered)")
```

Table 5: Balance within wave (name-type difference, wave fixed effects, firm-clustered)

variable	diff_D1_minus_D0	se	p_clustered
language_skills	-0.003	0.011	0.811
female	-0.001	0.014	0.963
ba_quality	0.015	0.012	0.227
extracurricular_skills	0.007	0.015	0.609
ma	-0.008	0.011	0.468
same_exp	0.000	0.009	0.977
exp_highquality	0.010	0.014	0.473

```
# Does the name penalty itself differ by wave?
ate_wave <- bind_rows(lapply(sort(unique(df_int1$fall_data)), function(k) {
  dat   <- filter(df_int1, fall_data == k)
  dat$y <- as.numeric(dat$callback)
  fit   <- lm_robust(y ~ D, data = dat, clusters = firmid, se_type = "CR2")
  data.frame(wave      = k,
             n         = nrow(dat),
             effect_pp = 100 * unname(coef(fit)["D"]),
             se_pp     = 100 * unname(fit$std.error["D"]),
             lo_pp     = 100 * unname(fit$conf.low["D"]),
             hi_pp     = 100 * unname(fit$conf.high["D"]))
}))

kable(ate_wave, digits = 2,
      caption = "Name penalty by wave (percentage points, firm-clustered CR2)")
```

Table 6: Name penalty by wave (percentage points, firm-clustered CR2)

wave	n	effect_pp	se_pp	lo_pp	hi_pp
0	1902	-4.36	1.17	-6.66	-2.06
1	1495	-4.79	1.36	-7.47	-2.11
2	2260	-4.29	1.11	-6.46	-2.13

3. test_calibration

The differential forest prediction is the number that matters. grf's p-value on it is one-sided (is the coefficient above zero?). Near 1 means well calibrated, between 0 and 1 means the forest overstates the spread, and at or below 0 means the spread isn't tracking real differences.

```
for (nm in names(forests)) {
  cat("\n====", nm, "====\n")
  print(test_calibration(forests[[nm]]$cf))
}
```

```
##
## ==== cf1 (3 moderators) ====
##
## Best linear fit using forest predictions (on held-out data)
## as well as the mean forest prediction as regressors, along
```

```
## with one-sided heteroskedasticity-robust (HC3) SEs:
##
##                                Estimate Std. Error t value    Pr(>t)
## mean.forest.prediction          1.00494    0.16788  5.9861 1.141e-09 ***
## differential.forest.prediction  0.16034    0.40485  0.3960     0.346
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## ==== cf1w (3 moderators + wave) ====
##
## Best linear fit using forest predictions (on held-out data)
## as well as the mean forest prediction as regressors, along
## with one-sided heteroskedasticity-robust (HC3) SEs:
##
##                                Estimate Std. Error t value    Pr(>t)
## mean.forest.prediction          0.99991    0.15874  6.2989 1.611e-10 ***
## differential.forest.prediction -0.18972    0.32274 -0.5878    0.7217
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## ==== cf2 (full covariates) ====
##
## Best linear fit using forest predictions (on held-out data)
## as well as the mean forest prediction as regressors, along
## with one-sided heteroskedasticity-robust (HC3) SEs:
##
##                                Estimate Std. Error t value    Pr(>t)
## mean.forest.prediction         0.994676   0.157125  6.3305 1.316e-10 ***
## differential.forest.prediction 0.057631   0.190464  0.3026    0.3811
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
cal_tab <- bind_rows(lapply(names(forests), function(nm) {
  tc <- test_calibration(forests[[nm]]$cf)
  data.frame(
    forest           = nm,
    mean_est         = tc["mean.forest.prediction", "Estimate"],
    mean_se          = tc["mean.forest.prediction", "Std. Error"],
    diff_est         = tc["differential.forest.prediction", "Estimate"],
    diff_se          = tc["differential.forest.prediction", "Std. Error"],
    diff_p_one_sided = tc["differential.forest.prediction", "Pr(>t)"]
  )
}))

kable(cal_tab, digits = 3, caption = "test_calibration results")
```

Table 7: test_calibration results

forest	mean_est	mean_se	diff_est	diff_se	diff_p_one_sided
cf1 (3 moderators)	1.005	0.168	0.160	0.405	0.346
cf1w (3 moderators + wave)	1.000	0.159	-0.190	0.323	0.722

forest	mean_est	mean_se	diff_est	diff_se	diff_p_one_sided
cf2 (full covariates)	0.995	0.157	0.058	0.190	0.381

4. RATE / TOC with a firm-level split

Half of the firms train a forest that produces the priority scores; the other half trains a separate evaluation forest. The split is by firm, not by resume, so resumes from one employer never sit on both sides. AUTOC is the primary measure and QINI the secondary one, chosen in advance. The priority score is the forest's predicted effect with no sign flip, so the top of the ranking is the group with the smallest predicted penalty (the least negative effects), and a positive RATE means the ranking sorts correctly.

Seed 1 is the headline split, fixed in advance. Seeds 2 to `n_seeds` are stability checks only. This chunk fits many forests and can take a few minutes.

```
Y_all <- as.numeric(df_int1$callback)
W_all <- df_int1$D
n_seeds <- 5

make_split <- function(firm, seed) {
  set.seed(seed)                        # set immediately before sampling so the split is reproducible
  f <- unique(firm)
  firm %in% sample(f, floor(length(f) / 2))
}

run_rate <- function(X, Y, W, firm, train, seed) {
  cf_train <- causal_forest(X[train, , drop = FALSE],  Y[train],  W[train],
                            clusters = firm[train],  seed = seed)
  cf_eval  <- causal_forest(X[!train, , drop = FALSE], Y[!train], W[!train],
                            clusters = firm[!train], seed = seed)
  priorities <- predict(cf_train, X[!train, , drop = FALSE])$predictions
  list(autoc = rank_average_treatment_effect(cf_eval, priorities, target = "AUTOC"),
       qini  = rank_average_treatment_effect(cf_eval, priorities, target = "QINI"))
}

rate_res <- lapply(forests, function(f) {
  out <- lapply(seq_len(n_seeds), function(s) {
    train <- make_split(df_int1$firmid, s)
    run_rate(f$X, Y_all, W_all, df_int1$firmid, train, seed = s)
  })
  names(out) <- seq_len(n_seeds)
  out
})

rate_tab <- bind_rows(lapply(names(rate_res), function(nm) {
  bind_rows(lapply(names(rate_res[[nm]]), function(s) {
    a <- rate_res[[nm]][[s]]$autoc
    q <- rate_res[[nm]][[s]]$qini
    data.frame(forest = nm, seed = as.integer(s), headline = (s == "1"),
               autoc = a$estimate, autoc_se = a$std.err,
               autoc_lo = a$estimate - 1.96 * a$std.err,
               autoc_hi = a$estimate + 1.96 * a$std.err,
               qini = q$estimate, qini_se = q$std.err)
```

```
  }))
}))

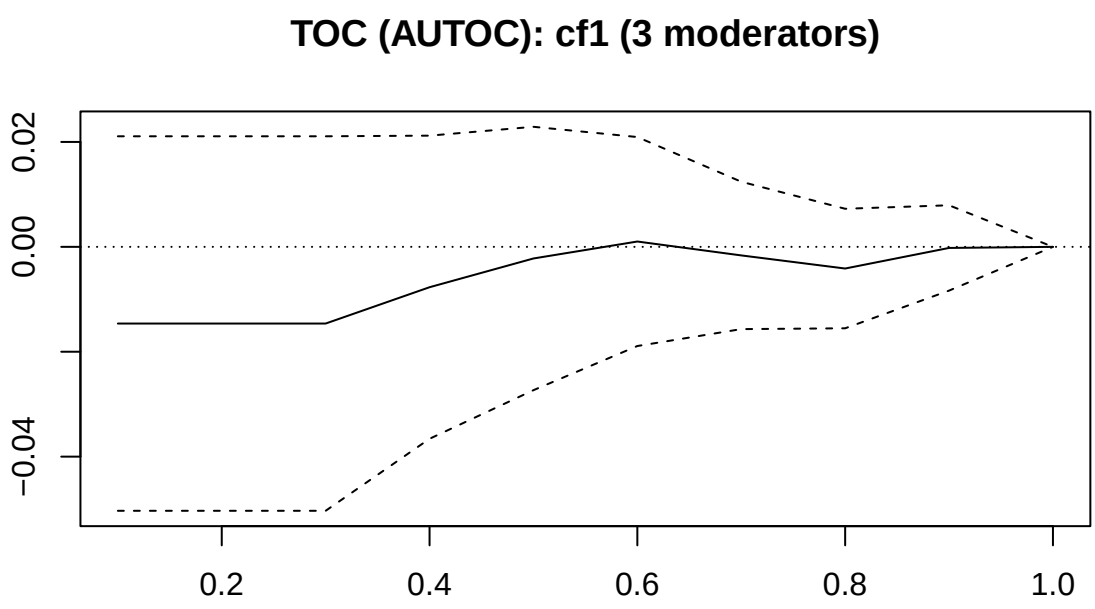
kable(rate_tab, digits = 4,
      caption = "RATE by forest and split seed (headline = seed 1)")
```

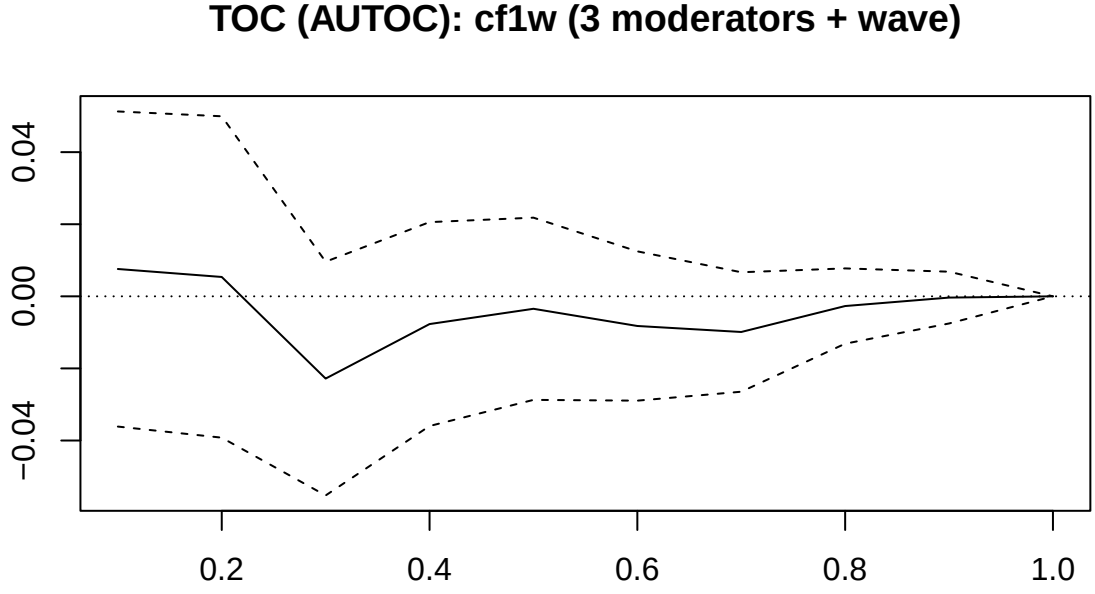
Table 8: RATE by forest and split seed (headline = seed 1)

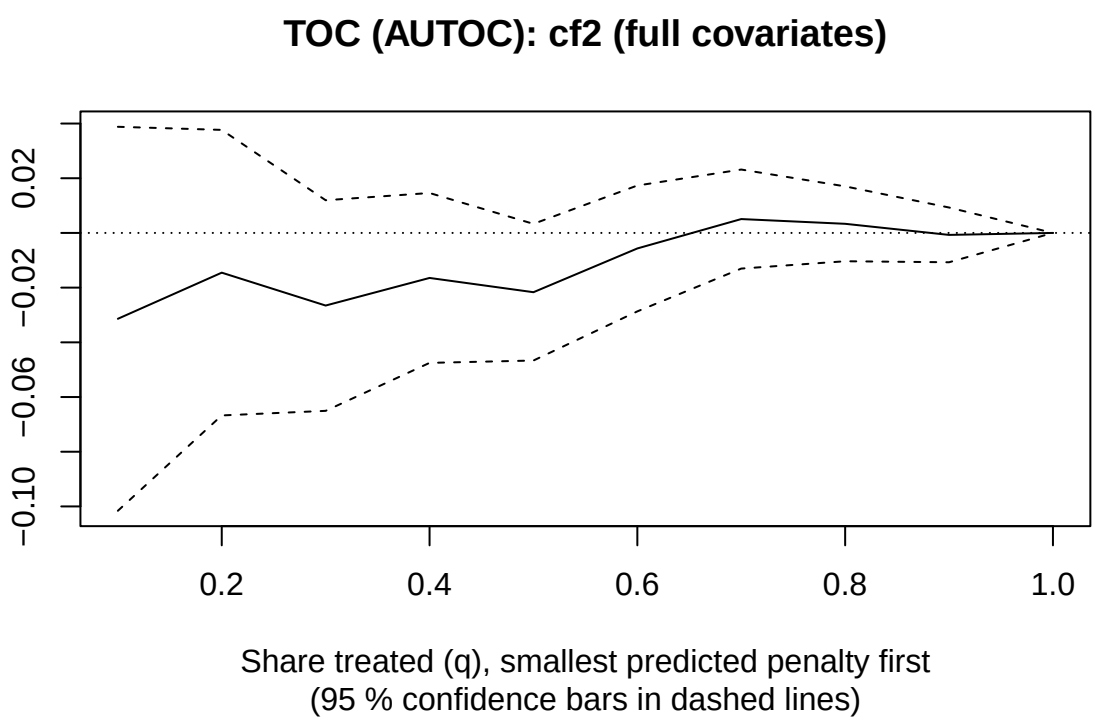
forest	seed	headline	autoc	autoc_se	autoc_lo	autoc_hi	qini	qini_se
cf1 (3 moderators)	1	TRUE	-0.0065	0.0105	-0.0271	0.0141	-0.0016	0.0037
cf1 (3 moderators)	2	FALSE	0.0120	0.0107	-0.0091	0.0331	0.0052	0.0034
cf1 (3 moderators)	3	FALSE	-0.0044	0.0107	-0.0253	0.0165	-0.0002	0.0037
cf1 (3 moderators)	4	FALSE	0.0065	0.0097	-0.0125	0.0254	0.0033	0.0032
cf1 (3 moderators)	5	FALSE	-0.0113	0.0128	-0.0363	0.0137	-0.0004	0.0033
cf1w (3 moderators + wave)	1	TRUE	-0.0048	0.0100	-0.0244	0.0149	-0.0024	0.0037
cf1w (3 moderators + wave)	2	FALSE	-0.0080	0.0103	-0.0282	0.0122	-0.0003	0.0030
cf1w (3 moderators + wave)	3	FALSE	-0.0100	0.0102	-0.0301	0.0100	-0.0025	0.0034
cf1w (3 moderators + wave)	4	FALSE	0.0118	0.0104	-0.0085	0.0321	0.0052	0.0034
cf1w (3 moderators + wave)	5	FALSE	-0.0092	0.0110	-0.0308	0.0125	-0.0012	0.0034
cf2 (full covariates)	1	TRUE	-0.0082	0.0129	-0.0334	0.0170	-0.0033	0.0039
cf2 (full covariates)	2	FALSE	-0.0154	0.0116	-0.0381	0.0073	-0.0051	0.0031
cf2 (full covariates)	3	FALSE	0.0089	0.0104	-0.0114	0.0292	0.0048	0.0037
cf2 (full covariates)	4	FALSE	0.0128	0.0115	-0.0097	0.0353	0.0036	0.0035
cf2 (full covariates)	5	FALSE	-0.0055	0.0135	-0.0320	0.0210	0.0010	0.0037

TOC curves for the headline split. The three-moderator forests can produce only a few distinct predictions, so many resumes are tied in the ranking and the curve will look stepped; mention this in the write-up.

```
for (nm in names(rate_res)) {
  plot(rate_res[[nm]][["1"]]$autoc, main = paste("TOC (AUTOC):", nm),
       xlab = "Share treated (q), smallest predicted penalty first")
}
```


TOC (AUTOC): cf1 (3 moderators)
0.02
0.00
−0.04
0.2
0.4
0.6
0.8
1.0
Share treated (q), smallest predicted penalty first
(95 % confidence bars in dashed lines)


TOC (AUTOC): cf1w (3 moderators + wave)
0.04
0.00
−0.04
0.2
0.4
0.6
0.8
1.0
Share treated (q), smallest predicted penalty first
(95 % confidence bars in dashed lines)



5. Combined reading (rule fixed in advance)

Both checks support heterogeneity: calibration differential coefficient positive and significant (one-sided $p < 0.05$) and the headline AUTOC interval excluding 0. Neither supports it: report that we find no statistically reliable evidence of heterogeneity, and do not interpret forest subgroup patterns. They disagree: call the evidence inconclusive and treat any subgroup patterns as exploratory only. A null here may reflect low power, not constant effects.

```
verdict_tab <- cal_tab %>%
  left_join(filter(rate_tab, headline), by = "forest") %>%
  mutate(
    calibration_supports = diff_p_one_sided < 0.05,
    rate_supports        = autoc_lo > 0,
    verdict = case_when(
      calibration_supports & rate_supports   ~ "Both support heterogeneity",
      !calibration_supports & !rate_supports ~ "Neither supports: report null",
      TRUE                                   ~ "Disagree: inconclusive"
    )
  ) %>%
  select(forest, diff_est, diff_se, diff_p_one_sided,
         autoc, autoc_lo, autoc_hi, verdict)

kable(verdict_tab, digits = 3, caption = "Validation verdict by forest")
```

Table 9: Validation verdict by forest

forest	diff_est	diff_se	diff_p_one_sided	autoc	autoc_lo	autoc_hi	verdict
cf1 (3 moderators)	0.160	0.405	0.346	-0.006	-0.027	0.014	Neither supports: report null
cf1w (3 moderators + wave)	-0.190	0.323	0.722	-0.005	-0.024	0.015	Neither supports: report null
cf2 (full covariates)	0.058	0.190	0.381	-0.008	-0.033	0.017	Neither supports: report null

6. Tuning robustness: one alternative setting (paste after the val-verdict chunk)

One alternative setting, fixed in advance: twice the default trees (4000 vs 2000) and a larger honesty fraction (0.7 vs the grf default 0.5). It is not a grid. The question is only whether the null verdict changes. Seed 1 is the same firm split as the headline RATE result, so the comparison is like for like.

```
alt <- list(num.trees = 4000, honesty.fraction = 0.7)

forests_alt <- lapply(forests, function(f) {
  list(X  = f$X,
       cf = causal_forest(f$X, Y_all, W_all,
                          clusters         = df_int1$firmid,
                          num.trees        = alt$num.trees,
                          honesty.fraction = alt$honesty.fraction,
                          seed             = 123))
})

cal_alt <- bind_rows(lapply(names(forests_alt), function(nm) {
  tc <- test_calibration(forests_alt[[nm]]$cf)
  data.frame(
    forest           = nm,
    diff_est         = tc["differential.forest.prediction", "Estimate"],
    diff_se          = tc["differential.forest.prediction", "Std. Error"],
    diff_p_one_sided = tc["differential.forest.prediction", "Pr(>t)"]
  )
}))
```

```
run_rate_alt <- function(X, Y, W, firm, train, seed, ...) {
  cf_train <- causal_forest(X[train, , drop = FALSE],  Y[train],  W[train],
                            clusters = firm[train],  seed = seed, ...)
  cf_eval  <- causal_forest(X[!train, , drop = FALSE], Y[!train], W[!train],
                            clusters = firm[!train], seed = seed, ...)
  priorities <- predict(cf_train, X[!train, , drop = FALSE])$predictions
  rank_average_treatment_effect(cf_eval, priorities, target = "AUTOC")
}

train_1 <- make_split(df_int1$firmid, 1)

rate_alt_tab <- bind_rows(lapply(names(forests), function(nm) {
  a <- run_rate_alt(forests[[nm]]$X, Y_all, W_all, df_int1$firmid, train_1, seed = 1,
                    num.trees        = alt$num.trees,
                    honesty.fraction = alt$honesty.fraction)
```

```
  data.frame(forest = nm,
             autoc    = a$estimate,
             autoc_lo = a$estimate - 1.96 * a$std.err,
             autoc_hi = a$estimate + 1.96 * a$std.err)
}))
```

```
stab_tab <- cal_tab %>%
  select(forest, diff_default = diff_est, p_default = diff_p_one_sided) %>%
  left_join(select(cal_alt, forest, diff_alt = diff_est, p_alt = diff_p_one_sided),
            by = "forest") %>%
  left_join(filter(rate_tab, headline) %>%
              select(forest, autoc_default = autoc,
                     lo_default = autoc_lo, hi_default = autoc_hi),
            by = "forest") %>%
  left_join(select(rate_alt_tab, forest, autoc_alt = autoc,
                   lo_alt = autoc_lo, hi_alt = autoc_hi),
            by = "forest")

kable(stab_tab, digits = 3,
      caption = "Default vs alternative tuning (4000 trees, honesty fraction 0.7)")
```

Table 10: Default vs alternative tuning (4000 trees, honesty fraction 0.7)

forest	diff_default	p_default	diff_alt	p_alt	autoc_default	lo_default	hi_default	autoc_alt	lo_alt	hi_alt
cf1 (3 moderators)	0.160	0.346	0.192	0.312	-0.006	-0.027	0.014	-0.006	-0.027	0.014
cf1w (3 moderators + wave)	-0.190	0.722	-0.209	0.746	-0.005	-0.024	0.015	-0.006	-0.026	0.014
cf2 (full covariates)	0.058	0.381	0.110	0.248	-0.008	-0.033	0.017	-0.009	-0.034	0.016

Reading it: the verdict is stable if every forest still has a calibration p above 0.05 and an AUTOC interval that includes 0 under the alternative setting. If one forest flips, report it as sensitive to tuning and treat the evidence for that forest as inconclusive under your combined rule.